SubTrack
Project Architecture & Revision Documentation

Code-grounded snapshot of the monolithic and multi-service implementations

Repository: chauhan12harsh/Subtrack
Revision baseline: 10 September 2026
Branches reviewed: main + monolithic-architecture

Purpose: This document is intended to be a restart manual. If the project is paused for 4–5 months, read this document first, then use the referenced source paths to re-enter the codebase quickly. It records both the intended architecture and important implementation details visible in the repository snapshot.

1. Executive Summary

SubTrack is a full-stack subscription-management application. The system covers authentication, role-based authorization, subscription lifecycle management, payments, notifications, and recurring renewal processing. The frontend is React + TypeScript. Two backend architectural forms exist:

- main: multi-service ASP.NET Core architecture with separate Authentication, Subscriptions, Payments, Notifications, and RenewalWorker projects.
- monolithic-architecture: one ASP.NET Core application containing the same major business capabilities internally, with feature separation through Controllers, Services, Interfaces, DTOs, Data, Entities, Enums, middleware, and migrations.
- Both versions use JWT authentication, role-based authorization, EF Core, SQL Server, and a similar business model. The major architectural difference is deployment/process boundaries and communication style.

2. Repository / Branch Map

Branch	Backend shape	Frontend	Primary purpose
main	Multiple ASP.NET Core projects + dedicated RenewalWorker	React 19 + TypeScript	Distributed/multi-service learning implementation
monolithic-architecture	Single ASP.NET Core Web API	React 19 + TypeScript	Monolith implementation and architectural comparison

Important repository note: GitHub reported that the two branches currently have no common ancestor. Therefore, treat them as two architectural codebases rather than assuming a normal branch diff is available.

3. Product Domain

The core domain can be remembered as five capabilities:

- Identity: registration, login, JWT issuance/validation, profile/password management, user and role administration.
- Subscriptions: create/update/read, status changes, categories, billing cycles, next billing date, due-subscription detection, renewal.
- Payments: user payments, worker/internal payments, payment lookup, subscription history, transaction history, admin visibility.
- Notifications: workflow notifications, current-user notifications, unread count, read/read-all, admin management.
- Renewal orchestration: identify due active subscriptions, attempt payment, notify the user, and renew on successful payment.

4. High-Level Architecture — Multi-Service

The main branch separates backend responsibilities into independently runnable projects:

Component	Role	Key interaction
Authentication Service	Identity, JWT, users/roles	Frontend + RenewalWorker
Subscription Service	Subscription lifecycle	Frontend + RenewalWorker
Payment Service	Payment processing/history	Frontend + RenewalWorker
Notification Service	Notifications/read state/admin operations	Frontend + RenewalWorker
RenewalWorker	Background recurring billing orchestration	HTTP calls to the four services

The repository README documents ports of 7025 (Authentication), 7081 (Subscriptions), 7070 (Payments), and 7056 (Notifications). The RenewalWorker is not a public API service; it is a BackgroundService.

5. High-Level Architecture — Monolith

The monolithic branch puts Authentication, Users, Subscriptions, Payments, Notifications, and worker-oriented operations inside one ASP.NET Core application. The boundaries remain visible in code, but requests do not cross network boundaries between these features.

Startup registration in Server/Server/Program.cs wires four EF Core contexts and five application services. JWT authentication, CORS, authentication/authorization middleware, and controllers are configured in the same process.

6. Request / Business Flow

Typical user flow:

1. React login/register → Authentication API → JWT returned.
2. React stores/uses the token for protected requests.
3. Subscription requests include the bearer token → authorization verifies identity/role → controller extracts the user ID → service executes business logic → EF Core persists/reads SQL Server.
4. Payment and notification features follow the same controller → service → EF Core pattern in the monolith.
5. In the multi-service version, the frontend calls individual APIs; the RenewalWorker calls the APIs over HTTP.

7. Automated Renewal Workflow

This is the most important business workflow to understand when restarting the project.

6. RenewalWorker wakes every 30 seconds.
7. If it has no JWT, it logs in using configured worker credentials.
8. It calls GET /subscription/due.
9. Only Active subscriptions whose NextBillingDate is today's UTC date are selected.
10. For each due subscription, the worker calls POST /payment/processinternal.
11. It creates a success/failure notification using POST /notification/create.
12. Only a successful payment triggers PATCH /subscription/renew/{id}.
13. Monthly subscriptions advance by one month; yearly subscriptions advance by one year.
14. The worker logs failures and continues processing rather than crashing the entire loop.

The monolith contains the same business operations as role-protected endpoints, but there is no separate deployable RenewalWorker process in that branch.

8. Authorization Model

Role	Meaning	Examples
User	Normal authenticated application user	Profile, own subscriptions, payments, notifications
Admin	System administrator	All users, roles, all subscriptions/payments/notifications
Worker	Internal renewal actor	Due subscriptions, internal payment, notification creation, renewal

The token contains the identity/role information used by ASP.NET Core authorization. In the monolith, SubscriptionController explicitly extracts ClaimTypes.NameIdentifier and parses it as a Guid.

9. Monolith Internal Layering

- Controllers: HTTP boundary, authorization attributes, route definitions, input handoff.
- Interfaces: contracts such as IAuthService, IUserService, ISubscriptionService, IPaymentService, INotificationService.
- Services: application/business logic and EF Core queries/updates.
- Data: EF Core DbContexts and persistence configuration.
- Entities: database/domain entities.
- DTOs: request/response contracts exposed to API clients.
- Enums: statuses, billing cycles, payment/notification types.
- Middlewares: cross-cutting HTTP concerns; the current main Authentication service explicitly registers GlobalExceptionMiddleware.
- Migrations: schema history for EF Core.

10. Subscription Rules Worth Remembering

- A duplicate subscription is rejected when the same user already has the same Name and Category.
- User subscription lists exclude Cancelled subscriptions and are ordered by NextBillingDate.

- Due detection uses DateTime.UtcNow.Date and matches Active subscriptions whose NextBillingDate.Date equals today.
- Renewal is allowed only for Active subscriptions.
- Monthly billing advances NextBillingDate with AddMonths(1).
- Yearly billing advances NextBillingDate with AddYears(1).
- A user can update only a subscription belonging to that user in the current service implementation.
- Admin deletion is separate from normal user subscription operations.

11. API Surface

The monolith README records 34 controller routes/actions. The main branch README records 30 endpoints across the four APIs. The important route groups are:

Area	Representative endpoints	Access
Auth	POST /auth/register, POST /auth/login	Public
Users	GET /user/profile, PATCH /user/update, PATCH /user/changepassword	Authenticated
Admin users	GET /user/alluser, PATCH /user/role/{userId}...	Admin
Subscriptions	POST /subscription/create, GET /subscription/user-subscription, PUT /subscription/status/...	Authenticated
Renewal	GET /subscription/due, PATCH /subscription/renew/{id}	Worker; due also Admin
Payments	POST /payment/process, GET /payment/transactions, GET /payment/subscription/{id}	Authenticated
Internal payment	POST /payment/processinternal	Worker
Notifications	GET /notification/my, PATCH /notification/readall, PATCH /notification/unreadcount	Authenticated
Notification workflow	POST /notification/create	Worker
Admin data	GET /.../all and notification/payment deletion	Admin

12. Frontend Structure

The React client is shared conceptually by both architectures. It contains pages for Login/Register, a dashboard layout, feature components for subscriptions, payments, notifications, and dedicated Axios service modules.

- Client/src/Pages — Login.tsx and Register.tsx.
- Client/src/Routes — AppRoute.tsx.
- Client/src/Layouts — DashboardLayout.tsx.
- Client/src/Components — Dashboard, Subscription, Payment, Notification, Sidebar, Budget and subscription modals.
- Client/src/Services/Auth — authApiClient.ts and authService.ts.
- Client/src/Services/Subscription, Payment, Notification — feature-specific API clients/services.
- Client/src/Types and Utils — frontend contracts and client-side helpers.

13. Database / Persistence Model

Both implementations use EF Core + SQL Server. The multi-service version uses service-specific DbContexts and migrations; the monolith also keeps multiple DbContexts inside one ASP.NET Core process. This preserves feature ownership boundaries without requiring multiple deployed applications.

For a future deep dive, start with the Data folders, then Entities, then Migrations. The database schema should be understood from the actual entities/configuration rather than inferred only from the API.

14. Configuration & Startup

Monolith startup responsibilities in Program.cs:

- Register controllers.
- Register UserDbContext, NotificationDbContext, PaymentDbContext and SubscriptionDbContext using the DefaultConnection.
- Configure enum serialization as strings.
- Configure JWT validation: issuer, audience, lifetime, signing key.
- Configure CORS from Cors:Origins.
- Register the five application services with scoped lifetime.
- Build pipeline: HTTPS redirection → CORS → authentication → authorization → controllers.

Multi-service startup follows the same ASP.NET Core pattern per service, but each service registers only its own DbContext and service implementations. The Authentication service additionally registers GlobalExceptionMiddleware.

15. What Changed Architecturally

Concern	Monolith	Multi-service	Why it matters
Deployment	One backend process	Multiple backend processes + worker	Operational complexity vs independent scaling
Communication	In-process service calls	HTTP between services	Latency, failures, contracts, retries become important
Database contexts	Multiple contexts in one app	Contexts belong to separate service projects	Clearer service ownership
Renewal	Operations exposed inside monolith	Dedicated BackgroundService	Distributed orchestration becomes explicit
Failure modes	Mostly local/in-process	Network and service failures	Requires resilience patterns
Security boundary	One application boundary	Each API has its own boundary	Service-to-service authentication matters
Scaling	Scale entire backend	Scale individual services/worker	Useful when workloads differ

16. Current Engineering Gaps / Future Work

This section is deliberately written as a restart checklist, not as a claim that every item is currently absent from every file. Before production-hardening, inspect and implement/verify:

- Idempotency for payment and renewal operations.
- Retries with bounded backoff for service-to-service HTTP calls.
- Timeouts and cancellation propagation for worker HTTP requests.
- Circuit breaker / resilience policies for dependent services.
- Distributed tracing and correlation IDs.
- Structured logging and centralized log aggregation.
- Global, consistent exception-to-HTTP response mapping across all services.
- Validation of DTOs and business rules at the API boundary.
- Concurrency control so the same due subscription cannot be renewed twice.
- Transactional consistency between payment success, subscription renewal, and notification creation.
- A durable job/queue model if polling every 30 seconds becomes insufficient.
- Secrets management rather than storing worker credentials/JWT secrets in ordinary configuration.
- Integration tests for the complete renewal workflow.
- Unit tests for subscription billing-cycle and status rules.
- CI/CD, deployment topology, health checks and observability.

17. Known Implementation Characteristics to Re-check

- RenewalWorker caches its JWT until the token is empty; there is no visible token-expiration refresh logic in Worker.cs.
- The worker catches exceptions per major operation and continues the 30-second loop.
- Due subscriptions are selected by UTC calendar date, so timezone/business-time requirements should be reconsidered for real billing.
- The worker performs payment, notification, and renewal as separate HTTP calls; there is no distributed transaction shown.
- The current renewal sequence can potentially be retried/re-entered, which is why idempotency/concurrency should be a priority hardening topic.
- The monolith uses multiple EF Core DbContexts but one SQL Server connection string in Program.cs; review whether this matches the intended database ownership model.

18. Restart Plan After 4–5 Months

Time	Action
10 minutes — Read this document	Reconstruct the domain, two architectures, roles, and renewal workflow.
10 minutes — Open main README	Confirm current high-level service boundaries and ports.
15 minutes — Open monolith Program.cs	Refresh dependency injection, DbContexts, JWT and middleware pipeline.
20 minutes — Trace SubscriptionController → SubscriptionService	Relearn the most important domain rules.
20 minutes — Trace Payment + Notification services	Understand transaction history and workflow outcomes.
20 minutes — Trace RenewalWorker	Follow token acquisition → due subscriptions → payment → notification → renewal.
20 minutes — Open frontend Services	Reconstruct how React talks to APIs and attaches authentication.
15 minutes — Run locally	Start backend/database/frontend and verify login +

	subscription CRUD.
30–60 minutes — Execute renewal manually	Create a due subscription in a safe local database and observe the full workflow.
After that — Choose architecture	Use monolith for simple development/debugging; use main for distributed-systems practice and service-level scaling/resilience.

19. Source-of-Truth File Map

Topic	Main branch	Monolith branch
Architecture overview	README.md	README.md
Startup	Server/<Service>/Program.cs	Server/Server/Program.cs
Subscription business logic	Server/Subscriptions/Services/SubscriptionService.cs	Server/Server/Services/SubscriptionService.cs
Subscription HTTP boundary	Server/Subscriptions/Controllers/SubscriptionController.cs	Server/Server/Controllers/SubscriptionController.cs
Renewal orchestration	Server/RenewalWorker/Worker.cs	No separate worker project
Worker startup	Server/RenewalWorker/Program.cs	N/A
Solution	Server/Substack.slnx	Server/Server project within solution
Frontend API integration	Client/src/Services/*	Client/src/Services/*
Database	Each service's Data/Entities/Migrations	Server/Server/Data + Entity + Migrations

20. Interview / Learning Mental Model

Be able to explain SubTrack in this order:

15. What problem does it solve? — Track subscriptions and automate recurring billing outcomes.
16. What are the bounded capabilities? — Identity, subscriptions, payments, notifications, renewal orchestration.
17. How does a request move? — Client → controller/API boundary → service → EF Core → SQL Server.
18. How is access controlled? — JWT authentication + User/Admin/Worker roles.
19. How does renewal work? — Worker discovers due subscriptions, charges, notifies, then advances billing date.
20. Why have two architectures? — To compare simple in-process modularity with distributed service boundaries.
21. What becomes harder in microservices? — Network failures, retries, idempotency, consistency, observability, deployment, service contracts.
22. What would you improve next? — Resilience, idempotency, concurrency, validation, testing, observability, and durable scheduling.

21. Revision Notes / Change Log

This document is a point-in-time snapshot. Whenever a major feature or architecture decision changes, update this document with: date, branch, decision, affected files, database impact, API changes, operational impact, and why the change was made.

Date	Branch	Change	Why / Impact
2026-09-10	main + monolithic-architecture	Initial architecture/revision documentation	Create restartable project knowledge base.

Appendix A — Quick Reference

- Frontend: React 19 + TypeScript + Vite + Axios + React Hook Form + React Router + Recharts + Tailwind CSS.
- Backend: ASP.NET Core; main branch targets .NET 10 in the Authentication project; monolith README documents .NET 10.
- Persistence: EF Core + SQL Server.
- Auth: JWT + role-based authorization.
- Roles: User / Admin / Worker.
- Core billing cycles: Monthly / Yearly.
- Core subscription statuses: Active / Paused / Cancelled.
- Renewal polling in main: every 30 seconds.
- Main service ports documented by README: Auth 7025, Subscription 7081, Payment 7070, Notification 7056.
- Primary development/testing aid: Postman collection.

Appendix B — Important Source Observations

The repository snapshot shows the multi-service SubscriptionService implementation and monolith SubscriptionService implementation as materially the same business logic, while the architectural packaging and startup boundaries differ. This is useful when explaining that the project is intentionally comparing deployment architecture rather than rewriting the domain from scratch.